Practice training

How we run a sprint

Two weeks to plan, build, show and reflect, and the report that tells everyone where we stand.

Where the gaps were

A scrum gap analysis from a team that went through this before us.

Pre-sprint	Sprint	Post-sprint
• Alignment document: epic definition and scope • Quarterly release goal planning • User story mapping • Release slicing • Story vertical slicing (INVEST)	• Definition of done • Definition of ready • Sprint goals • Budgets • Multiple boards, one team • Estimation gap • Spike usage	• New feature release notes

Most gaps were not about effort. They were about agreements nobody had written down: when a story is ready, when it is done, what the sprint is for.

Two weeks at a glance

Six sprints a quarter. Every sprint, the same five meetings.

Day 1

Sprint planning

Agree the sprint goal and the stories that get us there. Up to 2 hours.

Updates: New sprint report row: goal, stories committed

Every day

Standup

A short check on progress toward the sprint goal. The team decides its format.

Updates: Nothing on the roadmap

Mid-sprint

Refinement

Slice Deliverables into stories that meet the definition of ready. 1 hour.

Updates: Ready stories under each Deliverable

Last day

Sprint review

Show the work, celebrate wins, look honestly at misses. 1 hour.

Updates: Sprint report, commitment status, release notes

Last day

Retro

Reflect on how we work and pick one or two changes to try. 1 hour.

Updates: The team's working agreements

Who does what in a sprint

Product owner

- Writes user stories
- Writes acceptance criteria
- Carries the product vision into the backlog
- Orders the work
- Accepts finished stories

Development team

- Performs the work
- Adapts to change
- Breaks stories into tasks
- Sizes stories
- Ships working, deployable features

Scrum master

- Facilitates the meetings
- Coaches the team
- Removes impediments
- Gives feedback
- Helps write stories

The product manager is not in this picture by design. They set the roadmap and come to sprint review. Inside the sprint, the product owner speaks for the work.

The sprint goal

The single objective of the sprint. It says why the sprint is valuable to stakeholders and to the team.

What makes a sprint goal

- Decided at sprint planning
- The commitment of the sprint
- An immediate step toward a quarterly commitment
- Checked at every standup; the plan adapts to protect it
- Delivered within the sprint

Why have one

- Makes progress visible
- Gives focus, with room to flex on how
- Drives which stories come in and out
- Gets the team working together on one outcome

DMV Sprint 3 goal

"Field office staff can walk through the full renewal form in staging, from vehicle lookup to confirmation, with payment stubbed until legal approves the gateway." Goal met? Yes.

Sprint budgets

If a sprint is 10 days of work, 5 go to roadmap items. The rest are protected too.

Strategic: 5 days
Roadmap work. The product manager sets the order.

Technical: 2.5 days
Bugs first, then platform health. The technical architect orders it.

Small requests: 2.5 days
Kanban, first in, first out. The product owner can reorder.

Budgets are approximations

Use them at quarterly and sprint planning to keep a balance. A sprint can lean one way; the quarter should land near the split.

The split is set each quarter

50/25/25 is the default. The DMV team ran 60/20/20 in Q1 for a fixed launch date, and wrote down why.

The point: always leave room for the work that keeps the service healthy, and for the people who ask us for help.

Before the sprint: is the story ready?

A story comes into a sprint only when the team agrees it is ready.

Definition of ready

A story must meet INVEST and the team must agree it is ready.

I **Independent**
Can the work deliver value by itself, and are its dependencies known?

N **Negotiable**
Are the what and the why clear, with the how still open for discussion?

V **Valuable**
Is the value to users or stakeholders clear?

E **Estimable**
Do we have an idea of the complexity and effort?

S **Small**
Can we plan it into one sprint with decent certainty?

T **Testable**
Can we test that the work meets the acceptance criteria?

Our checklist adds

- Story format, linked to its Deliverable
- Acceptance criteria written as scenarios
- Refined at least once; more than one person agrees
- Design done, or not needed
- Accessibility and privacy needs named
- Security and dependencies cleared
- Rollout decided; help content flagged

One story, three versions

Version 1
Not ready

"As a customer, I want to be able to pay with credit cards so that I can purchase products online."

- Very vague: which cards? which payment process?
- Too broad: might take several sprints
- No acceptance criteria to test against

Version 2
Better, but not ready

"As a customer, I want to be able to pay using Visa or Mastercard credit cards, so that I can conveniently purchase products."

- More focused, but may depend on existing payment setup
- Smaller, but might still be large
- Testable only in part: no criteria for errors

Version 3
Ready

"As a customer shopping online, I want to pay using Visa or Mastercard, credit or debit, so that I have flexible payment options." Integrates with existing checkout; supports international cards.

- Can be built and released on its own
- Fits in a sprint, with backend in place
- Clear criteria: success, failure, card types, international

User story refinement

At least once a sprint, mid-sprint, one hour.

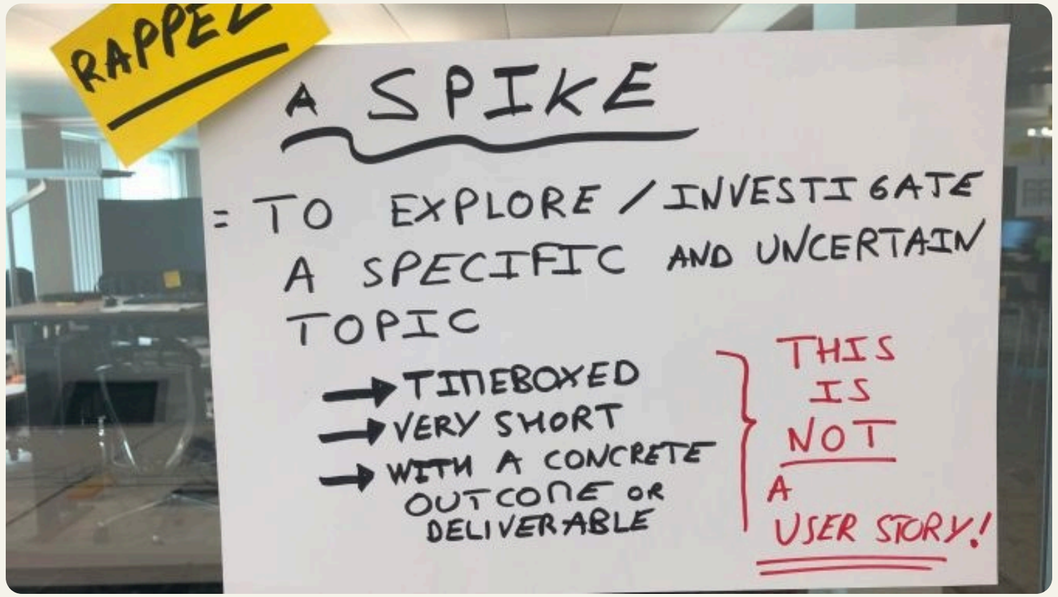
User story created	Team and PO refine	Team defines tasks	Work the story	Accept
The product owner writes the story.	Clearer acceptance criteria, and the story split if it is too big.	The team breaks the work into its task components.	The scrum master helps move it along as the team does the work.	The product owner accepts the story against the definition of done.

Who comes

Product owner, development team, UX designer and researcher, technical architect, scrum master.

What it produces

Ready stories under each Deliverable in the Now column, so sprint planning is a choice, not a writing session.



Spikes

When the team can't estimate a story because something is unknown, a spike answers one question first.

- Timeboxed to a few days
- One specific, uncertain question
- Ends in something concrete: a decision, an architecture decision record, or a prototype
- Not a user story

Our spike rule: a spike answers one question by a date. Without both, it's open-ended research.

During and after: is it done?

Done means deployable. If it can't go to production, it isn't done.

Definition of done

Why have one

- Everyone builds to the same bar
- Drives communication and teamwork
- Makes progress visible
- Reduces complexity
- Reduces technical debt

How to make one

- The whole scrum team writes it
- Review it at every retro
- Keep it simple and short
- Make it explicit
- Use it daily
- Include non-functional requirements

Ours

- Acceptance criteria met
- Code reviewed; tests pass
- Accessibility checked with keyboard and screen reader
- Security and non-functional needs met
- In production, or deployable behind a flag
- Release note and help content updated
- Product owner accepts

Sprint review

Last day of the sprint, one hour. Show the work, celebrate wins, look honestly at misses.

Who comes

Development team, product owner, scrum master, product manager. Stakeholders and the people who will use the work are invited to the demo.

What it updates

The sprint report: goal met, unplanned work, committed and completed, and the status of each quarterly commitment.

What ships

Release notes for anything that went to production, written for the person whose day changes.

DMV Sprint 3 review, Feb 14

Two field office staff saw the renewal form demo. Goal met, 11 of 12 stories done. Staff Training Program moved to Off track, and the product manager took it to the advisory and steering meetings that month instead of waiting for the quarter review.

The sprint report

Readable in five minutes: is the quarter on track, and what does the team need?

Sprint goal

One or two sentences

Goal met?

Yes or no

Committed / completed

Stories in, stories done

Unplanned work

What took real time and wasn't in the plan

Status per commitment

Committed, On track, Off track, Done

How to get back on track

For every Off track row

Status notes follow one pattern: percent complete; last, current and next action; timing; confidence. Bad news needs a why and a way back.

Example: an Off track row

DMV Sprint 3, Staff Training Program. Delivery type: hand off.

Status: **Off track**

10% complete. Slipped because materials were waiting for the finished service.

How to get back on track: start in parallel with the build, with one counter champion per pilot office using the beta early. Product manager decides after hearing the advisory group.

Without the change, training moves to early Q2.

"Off track" alone makes people nervous. "Off track, here is why, and here is our plan" makes them useful.

A release is not a launch

Keeping them apart lets us release small and often, while launches stay deliberate.

Release

Working software goes into production. A fix, a backend change, something switched off, or something only a beta group can see.

Nobody outside the team has to get ready.

Launch

The moment we expect people to start using something new.

Someone outside the team has to be ready: front-line staff, the help desk, content editors, the partner program.

Most releases are not launches. Every launch is built on releases that already happened.

Before every launch: are you ready?

The product owner sends a short go or no-go check to each group the launch affects.

One message per group

What is launching, to whom, and when. One question: are you ready? Reply yes, no, or with a concern by a set time.

Silence counts as yes

A group that doesn't reply takes responsibility for its own people.

A no pauses the launch

Not the release. The code can sit in production, switched off, until the group is ready.

Release notes go to the person whose day changes: internal notes before it goes live, external notes when it does. "You can now renew online in about five minutes" beats "Renewal module v1 deployed."

Enable: teach, then listen

After a launch, alternate between two modes.

Teach, train, help

- Guides and quick-reference sheets
- Hands-on training sessions
- Published help content
- Being available in the first weeks

Listen, capture, advocate

- Ask how it's going
- Watch usage data
- Collect stories from the front line
- Bring the themes back to the product team

Champions

Partner staff who learn the service early, help their peers, and report what they hear. They advocate; the product manager decides. Champions are not the help desk.

The test: can partners keep using the service after we move on, and can people succeed with it, not just reach it?

Retro

Last day, after the review, one hour. No managers unless the team asks.

Reflect

How did we work this sprint? What helped, what got in the way?

Pick one or two changes

Not ten. Try them next sprint and check at the next retro.

Review the agreements

Read the definitions of ready and done. Change them when they stop matching how we work.

The retro updates nothing on the roadmap. It updates how we work, which is what makes the next sprint better.

Rules of thumb

A sprint goal is an outcome, not a list.
One or two sentences. Yes or no at review.

Nothing comes in that isn't ready.
The scrum master guards the door; refinement keeps the queue full.

Done means deployable.
If it can't go to production, it isn't done.

Name the unplanned work.
It is usually why a number slipped.

Every Off track row gets a way back.
Cut scope, move dates, add help, or drop it.

Change one or two things each retro.
Practice is how the rest sticks.